

Table of Contents

MiniGui Framework Reference Manual

Platforma: Arduino, Raspberry Pi Pico

Graficka kniznica: U8G2

Cielovy displej: monochromaticke OLED displeje, napríklad SH1106 128x64

Zdrojove subory: MiniGui_with_Screen.h, MiniGui_with_Screen.cpp

Jazyk: C++ pre Arduino

Obsah

1. Uvod
 2. Rychly start
 3. Architektura frameworku
 4. UML diagram tried
 5. Event system
 6. Stav widgetov: visible, selectable, selected, editing
 7. Callback system
 8. Invalidation a redraw system
 9. Trieda View
 10. Trieda Group
 11. Trieda Screen
 12. Trieda ScreenManager
 13. Trieda Label
 14. Trieda Icon
 15. Trieda CheckButton
 16. Trieda Number
 17. Trieda List
 18. Trieda Pages
 19. MessageBox
 20. Tvorba vlastneho widgetu
 21. Odporucana struktura Battery Tester aplikacie
 22. Kompletne priklady
 23. Troubleshooting
 24. API referencia v skratke
-

1. Uvod

MiniGui je jednoduchý objektový GUI framework pre malé embedded zariadenia. Je navrhnutý pre Arduino prostredie, Raspberry Pi Pico a grafickú knižnicu U8G2. Framework používa statický model objektov: widgety sa vytvárajú ako globálne alebo statické objekty a do skupín sa vkladajú cez pole pointerov. Nepoužíva dynamickú alokáciu pamäte typu new alebo malloc, čo je výhodné pre stabilitu embedded aplikácií.

Framework je vhodný najmä pre zariadenia ovládané rotačným encoderom s tlačidlom. Udalosti sú reprezentované enumom Event a posielajú sa do aktívnej obrazovky alebo skupiny widgetov.

Hlavné vlastnosti:

- widgety odvodené od spoločnej triedy View,
 - kontajner Group s výberom podriadených widgetov,
 - obrazovky Screen a správca obrazoviek ScreenManager,
 - modálne okno MessageBox,
 - stránkovací widget Pages,
 - callback pri stlačení tlačidla,
 - rozlíšenie stavov selected a editing,
 - invalidation/redraw systém pre prekreslenie iba vtedy, keď je to potrebné.
-

2. Rychlý start

2.1 Subory v projekte

Do Arduino projektu vlož:

MiniGui.h
MiniGui.cpp
YourSketch.ino

Ak používate aktuálnu verziu s obrazovkami, môžete subory premenovať z:

MiniGui_with_Screen.h -> MiniGui.h
MiniGui_with_Screen.cpp -> MiniGui.cpp

V MiniGui.cpp musí byť include zhodný s názvom hlavičky:

```
#include "MiniGui.h"
```

2.2 Minimalný program

```
#include <Arduino.h>  
#include <U8g2lib.h>  
#include "MiniGui.h"
```

```

using namespace MiniGui;

U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0);

Label title(0, 12, "Battery Tester", u8g2_font_6x10_tf);
CheckBox enable(0, 20, false);
Number voltage(20, 18, 40, 14, 12, 0, 24, u8g2_font_6x10_tf);

View* mainViews[] = {
    &title,
    &enable,
    &voltage
};

Screen mainScreen("Main", mainViews, 3, false);
ScreenManager screenManager(u8g2);

bool readEncoderEvent(Event& event) {
    return false; // dopln realne citanie encodera
}

void setup() {
    u8g2.begin();
    screenManager.setScreen(&mainScreen);
}

void loop() {
    Event event;

    if (readEncoderEvent(event)) {
        screenManager.handleEvent(event);
    }

    screenManager.update();
}

```

3. Architektura frameworku

Zakladom frameworku je trieda View. Kazdy vykreslitelny prvok dedi od View. Kontajner Group dedi od View a spravuje pole deti. Screen dedi od Group a reprezentuje jednu celu obrazovku. ScreenManager drzi aktualnu obrazovku, posila jej udalosti a prekresluje ju pri invalidacii.

View

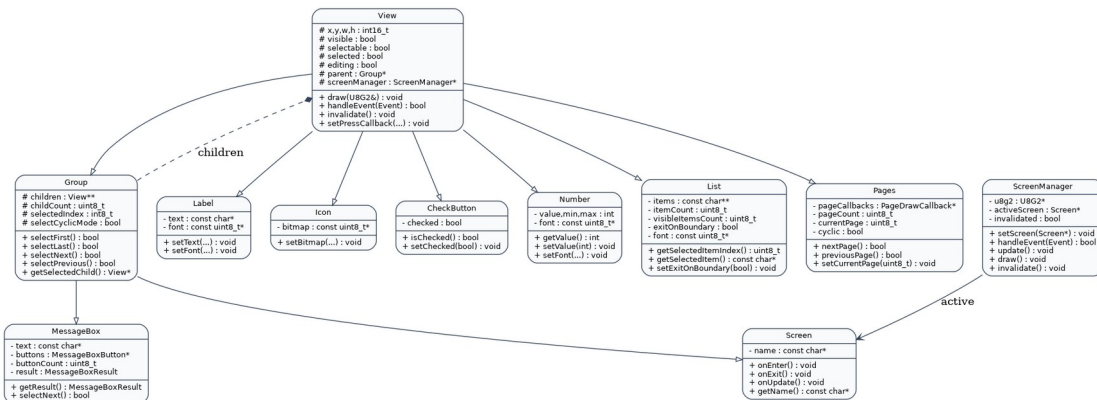
- Group
 - Screen
 - MessageBox
- Label
- Icon
- CheckBox
- Number
- List
- Pages

Tok udalosti:

encoder -> Event -> ScreenManager -> active Screen -> selected child -> pripadne Group

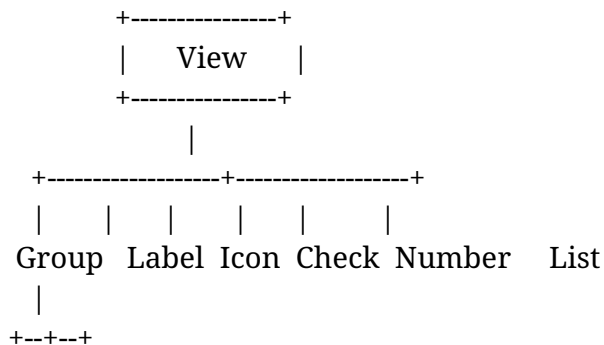
Ak vybrany widget udalost spracuje, vrati true. Ak ju nespracuje, vrati false a nadradena skupina sa moze pokusit udalost spracovat sama. Toto je jednoduchá forma event bubblingu.

4. UML diagram tried



MiniGui UML

Textova verzia:



| |
Screen MessageBox

View -> Pages

ScreenManager -> Screen

Group -> View[]

5. Event system

5.1 Enum Event

```
enum class Event {  
    TURN_LEFT,  
    TURN_RIGHT,  
    PRESS_BUTTON,  
    LONG_HOLD_BUTTON  
};
```

Vyznamen udalosti:

Event	Vyznamenanie
TURN_LEFT	encoder sa otocil dolava alebo na predchadzajucu hodnotu
TURN_RIGHT	encoder sa otocil doprava alebo na nasledujucu hodnotu
PRESS_BUTTON	kratke stlacenie tlacidla encoderu
LONG_HOLD_BUTTON	dlhe podrzanie tlacidla

5.2 Spracovanie udalosti

Kazdy widget ma metodu:

```
virtual bool handleEvent(Event event);
```

Vracia:

- true: udalost bola spracovana,
- false: udalost nebola spracovana a moze ju spracovat nadradena trieda.

5.3 Priklad citania eventov

```
bool readEncoderEvent(Event& event) {  
    if (encoderMovedLeft()) {  
        event = Event::TURN_LEFT;  
        return true;  
    }  
}
```

```

if (encoderMovedRight()) {
    event = Event::TURN_RIGHT;
    return true;
}

if (buttonPressed()) {
    event = Event::PRESS_BUTTON;
    return true;
}

return false;
}

```

6. Stavby widgetov: visible, selectable, selected, editing

Kazdy View obsahuje tieto stavove premenne:

Stav	Vyznamenanie
visible	widget sa kresli a moze byt aktivny
selectable	widget moze byt vybrany encoderom
selected	widget je aktualne vybrany
editing	widget je v rezime editacie hodnoty

6.1 Selected vs editing

selected znamena, ze widget ma fokus. Typicky sa zobrazi ramikom.

editing znamena, ze widget priamo meni svoju hodnotu. Typicky sa zobrazi inverzne.

Priklad pre Number:

selected = true, editing = false -> hodnota je oramovana

selected = true, editing = true -> hodnota je inverzne zobrazena a encoder ju meni

7. Callback system

Callback sa vola pri prijati udalosti PRESS_BUTTON.

7.1 Typ callbacku

```

typedef void (*PressCallback)(View* sender);

```

Callback dostane pointer na widget, ktory udalost vyvolal.

7.2 Nastavenie callbacku

```
void onVoltagePressed(View* sender) {  
    Number* number = (Number*)sender;  
    int value = number->getValue();  
}
```

```
voltage.setPressCallback(onVoltagePressed);
```

7.3 Použitie s List

```
void onMenuPressed(View* sender) {  
    List* menu = (List*)sender;  
  
    switch (menu->getSelectedItemIndex()) {  
        case 0:  
            // Start  
            break;  
        case 1:  
            // Settings  
            break;  
    }  
}
```

8. Invalidation a redraw system

Invalidation system riesi, aby sa displej neprekresloval neustale, ale len pri zmene.

8.1 Zakladny princip

ScreenManager obsahuje priznak:

```
bool invalidated = true;
```

Ak sa zmeni hodnota widgetu, zavola sa:

```
invalidate();
```

Ta zavola screenManager->invalidate() a pri najblizsom screenManager.update() sa obrazovka prekresli.

8.2 Typicka slucka

```
void loop() {  
    Event event;  
  
    if (readEncoderEvent(event)) {  
        screenManager.handleEvent(event);  
    }  
}
```

```
}  
  
screenManager.update();  
}
```

8.3 Kedy volat invalidate()

Volaj invalidate() pri:

- zmene textu,
- zmene hodnoty,
- zmene stavu checked,
- zmene aktualnej stranky,
- zmene obrazovky,
- prijati novej hodnoty zo senzora.

Poznámka: v aktualnej implementacii niektore settery invalidáciu este nemusia volat. Ak chces automaticke prekreslenie po kazdom setteri, dopln invalidate() do setterov ako setText, setChecked, setValue a podobne.

9. Trieda View

View je abstraktna zakladna trieda vsetkych widgetov.

9.1 Konstruktory

View(int16_t x, int16_t y, int16_t w, int16_t h, bool selectable = false);

Parametre:

Parameter	Popis
x	X pozicia v pixeloch
y	Y pozicia v pixeloch
w	sirka v pixeloch
h	vyska v pixeloch
selectable	ci moze byt widget vybrany

9.2 draw

virtual void draw(U8G2& u8g2) = 0;

Cisto virtualna metoda. Kazdy potomok musi implementovat vlastne vykreslenie.

9.3 handleEvent

virtual bool handleEvent(Event event);

Zakladna implementacia spracuje PRESS_BUTTON, ak je widget selectable, a zavola notifyPress().

9.4 Rodic a ScreenManager

```
void setParent(Group* parent);  
Group* getParent() const;
```

```
virtual void setScreenManager(ScreenManager* manager);  
ScreenManager* getScreenManager() const;  
void invalidate();
```

parent sluzi na hierarchiu widgetov. screenManager sluzi na redraw system.

9.5 Viditelnost

```
void setVisible(bool value);  
bool isVisible() const;
```

isVisible() vracia aktualny stav viditelnosti.

9.6 Vyberatelnost

```
void setSelectable(bool value);  
bool isSelectable() const;
```

isSelectable() vracia selectable && visible. Neviditelny widget sa teda nebude vyberat.

9.7 Stav selected

```
void setSelected(bool value);  
bool isSelected() const;
```

9.8 Stav editing

```
void setEditing(bool value);  
bool isEditing() const;
```

9.9 Callback

```
void setPressCallback(PressCallback callback);  
void notifyPress();
```

9.10 Geometria

```
int16_t getX() const;  
int16_t getY() const;  
int16_t getW() const;  
int16_t getH() const;
```

10. Trieda Group

Group je kontajner, ktorý drží pole pointerov na View objekty.

10.1 Konštruktor

```
Group(  
    int16_t x,  
    int16_t y,  
    int16_t w,  
    int16_t h,  
    View** children,  
    uint8_t childCount,  
    bool selectCyclicMode = true  
);
```

Parametre:

Parameter	Popis
children	pole pointerov na deti
childCount	pocet deti
selectCyclicMode	cyklicky vyber

10.2 draw

```
void draw(U8G2& u8g2) override;
```

Vykresli všetky viditeľné deti.

10.3 handleEvent

```
bool handleEvent(Event event) override;
```

Najprv pošle udalosť vybranému dietatu. Ak ju dieta nespracuje, Group spracuje TURN_LEFT alebo TURN_RIGHT ako presun vyberu.

10.4 Vybranie dieťaťa

```
View* getSelectedChild();  
int8_t getSelectedIndex() const;
```

10.5 Vyber deti

```
virtual bool selectFirst();  
virtual bool selectLast();  
virtual bool selectNext();  
virtual bool selectPrevious();
```

Všetky vracajú:

- true, ak sa podarilo vybrať platný selectable widget,

- false, ak taka položka neexistuje.

10.6 Nastavenie indexu

```
bool setSelectedIndex(uint8_t index);
```

Nastavi vybrany widget na zadany index, iba ak index existuje a widget je selectable.

10.7 Cyklicky vyber

```
void setSelectCyclicMode(bool value);
```

```
bool getSelectCyclicMode() const;
```

Ak je selectCyclicMode == true, po poslednej položke nasleduje prvá a pred prvou nasleduje posledná.

Ak je selectCyclicMode == false, selectNext() na konci vráti false a event môže spracovať nadradená skupina.

10.8 Propagácia ScreenManagera

```
void setScreenManager(ScreenManager* manager) override;
```

Nastavi manager aj všetkým deťom.

11. Trieda Screen

Screen reprezentuje jednu celú obrazovku. Dedi od Group.

11.1 Konštruktor pre celú obrazovku

```
Screen(  
    const char* name,  
    View** children,  
    uint8_t childCount,  
    bool selectCyclicMode = false  
);
```

Používa rozmery 0, 0, 128, 64.

11.2 Konštruktor s rozmermi

```
Screen(  
    const char* name,  
    int16_t x,  
    int16_t y,  
    int16_t w,  
    int16_t h,  
    View** children,  
    uint8_t childCount,
```

```
    bool selectCyclicMode = false  
);
```

11.3 Nazov obrazovky

```
const char* getName() const;
```

11.4 Lifecycle metody

```
virtual void onEnter();
```

```
virtual void onExit();
```

```
virtual void onUpdate();
```

- onEnter() sa vola po aktivacii obrazovky.
- onExit() sa vola pred opustenim obrazovky.
- onUpdate() sa vola pri kazdom ScreenManager::update().

11.5 Odvodena obrazovka

```
class MeasureScreen : public Screen {
```

```
public:
```

```
    MeasureScreen(View** children, uint8_t count)  
        : Screen("Measure", children, count, false) {}
```

```
    void onEnter() override {  
        // reset merania  
    }
```

```
    void onUpdate() override {  
        // citanie serialu alebo merania  
    }  
};
```

12. Trieda ScreenManager

ScreenManager riadi aktivnu obrazovku a prekreslovanie.

12.1 Konstruktor

```
ScreenManager(U8G2& u8g2);
```

12.2 Aktivna obrazovka

```
void setScreen(Screen* screen);
```

```
Screen* getActiveScreen() const;
```

setScreen() zavola onExit() na starej obrazovke, nastavi novu, propaguje ScreenManager do jej deti, zavola onEnter() a invaliduje displej.

12.3 Spracovanie udalosti

```
bool handleEvent(Event event);
```

Posle udalost aktivnej obrazovke. Ak bola spracovana, invaliduje obrazovku.

12.4 Prekreslenie

```
void draw();
```

```
void update();
```

draw() vymaze buffer, vykresli aktivnu obrazovku a zavola sendBuffer().

update() zavola activeScreen->onUpdate() a ak je invalidated == true, zavola draw().

12.5 Invalidation

```
void invalidate();
```

```
bool isInvalidated() const;
```

13. Trieda Label

Label vykresluje pevny text.

13.1 Konstruktor

```
Label(int16_t x, int16_t y, const char* text, const uint8_t* font);
```

x a y su suradnice baseline textu, tak ako pri u8g2.drawStr().

13.2 Text

```
void setText(const char* text);
```

```
const char* getText() const;
```

13.3 Font

```
void setFont(const uint8_t* font);
```

```
const uint8_t* getFont() const;
```

Pred vykreslenim sa zavola u8g2.setFont(font), ak font != nullptr.

13.4 draw

```
void draw(U8G2& u8g2) override;
```

14. Trieda Icon

Icon vykresluje bitmapu pomocou u8g2.drawXBMP().

14.1 Konštruktor

```
Icon(int16_t x, int16_t y, int16_t w, int16_t h, const uint8_t* bitmap, bool selectable = false);
```

14.2 Bitmapa

```
void setBitmap(const uint8_t* bitmap);
```

14.3 draw

```
void draw(U8G2& u8g2) override;
```

Ak je selected, ikona sa oramuje. Ak je editing, pozadie sa vykresli inverzne.

15. Trieda CheckButton

CheckButton je jednoduché zaskrtavacie políčko.

15.1 Konštruktor

```
CheckButton(int16_t x, int16_t y, bool checked = false);
```

15.2 Stav

```
bool isChecked() const;
```

```
void setChecked(bool value);
```

15.3 Eventy

Event	Správanie
PRESS_BUTTON	prepne checked, zavola callback, vrati true
ostatne	vrati false

15.4 Příklad

```
CheckButton outputEnabled(4, 16, false);
```

```
void onOutputPressed(View* sender) {  
    CheckButton* cb = (CheckButton*)sender;  
    if (cb->isChecked()) {  
        // output ON  
    }  
}
```

16. Trieda Number

Number je editovateľné celé číslo.

16.1 Kontruktor

```
Number(  
    int16_t x,  
    int16_t y,  
    int16_t w,  
    int16_t h,  
    int value,  
    int minValue,  
    int maxValue,  
    const uint8_t* font  
);
```

16.2 Hodnota

```
int getValue() const;  
void setValue(int value);
```

setValue() oreze hodnotu na interval <minValue, maxValue>.

16.3 Font

```
void setFont(const uint8_t* font);  
const uint8_t* getFont() const;
```

16.4 Eventy

Event	Spravanie
PRESS_BUTTON	prepne editing, zavola callback
TURN_LEFT	ak editing, znizi hodnotu
TURN_RIGHT	ak editing, zvyysi hodnotu
LONG_HOLD_BUTTON	ukonci editing

16.5 Vizualizacia

- selected == true a editing == false: hodnota je oramovana.
 - editing == true: hodnota je inverzne vykreslena.
-

17. Trieda List

List je posuvatelny zoznam textovych poloziek.

17.1 Kontruktor

```
List(  
    int16_t x,  
    int16_t y,  
    int16_t w,  
    int16_t h,
```

```

const char** items,
uint8_t itemCount,
uint8_t visibleItemsCount,
const uint8_t* font
);

```

17.2 Vybrana položka

```

uint8_t getSelectedItemIndex() const;
const char* getSelectedItem() const;

```

17.3 Exit on boundary

```

void setExitOnBoundary(bool value);
bool getExitOnBoundary() const;

```

Ak `exitOnBoundary == true`:

- na prvej položke `TURN_LEFT` vrati `false`,
- na poslednej položke `TURN_RIGHT` vrati `false`.

Tým môže udalosť spracovať nadradený Group, a používateľ môže zo zoznamu vysociť bez špeciálneho tlačidla.

17.4 Font

```

void setFont(const uint8_t* font);
const uint8_t* getFont() const;

```

17.5 Eventy

Event	Správanie
<code>TURN_LEFT</code>	posun na predchádzajúcu položku alebo exit
<code>TURN_RIGHT</code>	posun na nasledujúcu položku alebo exit
<code>PRESS_BUTTON</code>	zavola callback

18. Trieda Pages

Pages je widget na vykresľovanie viacerých stránok. Každá stránka má vlastný callback.

18.1 Callback

```

typedef void (*PageDrawCallback)(U8G2& u8g2, uint8_t pageIndex);

```

Callback dostane `U8G2&` a index aktuálnej stránky.

18.2 Kontruktor

```
Pages(  
    int16_t x,  
    int16_t y,  
    int16_t w,  
    int16_t h,  
    PageDrawCallback* pageCallbacks,  
    uint8_t pageCount,  
    bool cyclic = false,  
    bool selectable = true  
);
```

18.3 Aktualna stranka

```
uint8_t getCurrentPage() const;  
uint8_t getPageCount() const;  
void setCurrentPage(uint8_t page);
```

18.4 Navigacia

```
bool nextPage();  
bool previousPage();
```

Vracaju true, ak sa stranka zmenila, inak false.

18.5 Cyklicky rezim

```
void setCyclic(bool value);  
bool getCyclic() const;
```

18.6 Navigacne sipky

Po vykresleni stranky Pages automaticky dokresli lavu a pravu sipku, ak existuje predchadzajuca alebo nasledujuca stranka. Sipky sa kreslia cez `u8g2.drawTriangle()`.

18.7 Priklad

```
void drawInfoPage(U8G2& u8g2, uint8_t pageIndex) {  
    u8g2.setFont(u8g2_font_6x10_tf);  
  
    if (pageIndex == 0) {  
        u8g2.drawStr(18, 20, "U = 12.4 V");  
    } else if (pageIndex == 1) {  
        u8g2.drawStr(18, 20, "I = 0.52 A");  
    }  
}
```

```
PageDrawCallback callbacks[] = {  
    drawInfoPage,
```

```
drawInfoPage  
};
```

```
Pages pages(0, 0, 128, 64, callbacks, 2, false, true);
```

Poznamka: v tomto priklade obidve polozky pola ukazuju na rovnaku funkciu a rozlisuju sa pomocou pageIndex. Mozes vsak pouzit aj samostatnu funkciu pre kazdu stranku.

19. MessageBox

MessageBox je modalne okno s textom a tlacidlami.

19.1 Vysledok

```
enum class MessageBoxResult {  
    NONE = 0,  
    YES,  
    NO,  
    OK,  
    CANCEL,  
    CUSTOM  
};
```

19.2 Typ tlacidla

```
enum class MessageBoxButtonType {  
    TEXT,  
    ICON  
};
```

19.3 Struktura tlacidla

```
struct MessageBoxButton {  
    MessageBoxResult result;  
    MessageBoxButtonType type;  
    const char* text;  
    const uint8_t* icon16x16;  
};
```

19.4 Konstruktor

```
MessageBox(  
    int16_t x,  
    int16_t y,  
    int16_t w,  
    int16_t h,  
    const char* text,  
    MessageBoxButton* buttons,
```

```

uint8_t buttonCount,
const uint8_t* font
);

```

19.5 handleEvent

Event	Spravanie
TURN_LEFT	vyber predchadzajuce tlacidlo
TURN_RIGHT	vyber nasledujuce tlacidlo
PRESS_BUTTON	nastavi result podla tlacidla
LONG_HOLD_BUTTON	nastavi CANCEL

19.6 Blokujuca funkcia messageBox

```

MessageBoxResult messageBox(
    U8G2& u8g2,
    const char* text,
    MessageBoxButton* buttons,
    uint8_t buttonCount,
    EventReader eventReader,
    const uint8_t* font
);

```

Funkcia vytvori modalne okno, cita eventy cez eventReader, vykresluje okno a skonci po vybere tlacidla.

19.7 Priklad Ano/Nie

```

MessageBoxButton buttons[] = {
    { MessageBoxResult::YES, MessageBoxButtonType::TEXT, "Ano", nullptr },
    { MessageBoxResult::NO, MessageBoxButtonType::TEXT, "Nie", nullptr }
};

```

```

MessageBoxResult r = messageBox(
    u8g2,
    "Spustit test?",
    buttons,
    2,
    readEncoderEvent,
    u8g2_font_6x10_tf
);

```

```

if (r == MessageBoxResult::YES) {
    // start test
}

```

20. Tvorba vlastneho widgetu

Vlastny widget odvod od View.

```
class BatteryGauge : public View {
private:
    uint8_t percent;

public:
    BatteryGauge(int16_t x, int16_t y, int16_t w, int16_t h)
        : View(x, y, w, h, false), percent(0) {}

    void setPercent(uint8_t value) {
        if (value > 100) value = 100;
        percent = value;
        invalidate();
    }

    void draw(U8G2& u8g2) override {
        u8g2.drawFrame(x, y, w, h);
        int fill = (w - 2) * percent / 100;
        u8g2.drawBox(x + 1, y + 1, fill, h - 2);
    }
};
```

Ak widget potrebuje reagovat na encoder, implementuj `handleEvent()`.

21. Odporucana struktura Battery Tester aplikacie

Pre battery tester odporucana sada obrazoviek:

MainScreen
SettingsScreen
MeasureScreen
ResultScreen
CalibrationScreen
InfoScreen

21.1 MainScreen

Obsahuje hlavne menu:

- Start test,
- Settings,
- Calibration,

- Info.

21.2 SettingsScreen

Obsahuje Number widgety:

- max current,
- reference current,
- timeout,
- pocet vzoriek.

21.3 MeasureScreen

Obsahuje:

- stav merania,
- aktualne U/I,
- Pages pre viac meracich zobrazeni,
- MessageBox pre potvrdenie stop.

21.4 ResultScreen

Obsahuje:

- Ue,
 - Ri,
 - R^2 ,
 - RMSE,
 - graf alebo viac stranok vysledkov cez Pages.
-

22. Kompletne priklady

22.1 Menu so ScreenManagerom

```
const char* mainItems[] = {
    "Start",
    "Settings",
    "Info"
};
```

```
List mainMenu(0, 16, 128, 36, mainItems, 3, 3, u8g2_font_6x10_tf);
```

```
Label title(0, 10, "Battery Tester", u8g2_font_6x10_tf);
```

```
View* mainViews[] = {
    &title,
    &mainMenu
};
```

```

};

Screen mainScreen("Main", mainViews, 2, false);
ScreenManager screenManager(u8g2);

void onMainMenuPressed(View* sender) {
    List* list = (List*)sender;

    if (list->getSelectedItemIndex() == 1) {
        screenManager.setScreen(&settingsScreen);
    }
}

void setup() {
    u8g2.begin();
    mainMenu.setPressCallback(onMainMenuPressed);
    screenManager.setScreen(&mainScreen);
}

```

22.2 Settings obrazovka

```

Label settingsTitle(0, 10, "Settings", u8g2_font_6x10_tf);
Number maxCurrent(0, 20, 50, 14, 500, 0, 2000, u8g2_font_6x10_tf);
Number timeout(60, 20, 50, 14, 10, 1, 60, u8g2_font_6x10_tf);

```

```

View* settingsViews[] = {
    &settingsTitle,
    &maxCurrent,
    &timeout
};

```

```

Screen settingsScreen("Settings", settingsViews, 3, false);

```

22.3 Pages pre vysledky

```

void drawResultPage(U8G2& u8g2, uint8_t page) {
    u8g2.setFont(u8g2_font_6x10_tf);

    if (page == 0) {
        u8g2.drawStr(18, 18, "Ue: 12.54 V");
        u8g2.drawStr(18, 30, "Ri: 0.082 Ohm");
    } else if (page == 1) {
        u8g2.drawStr(18, 18, "R2: 0.998");
        u8g2.drawStr(18, 30, "RMSE: 0.012");
    }
}

```

```
PageDrawCallback resultPages[] = {  
    drawResultPage,  
    drawResultPage  
};
```

```
Pages resultPagesWidget(0, 0, 128, 64, resultPages, 2, false, true);
```

23. Troubleshooting

Encoder nereaguje

Skontroluj:

- ci readEncoderEvent() vracia true,
- ci nastavuje spravny Event,
- ci volas screenManager.handleEvent(event),
- ci mas aktivnu obrazovku cez setScreen().

Widget sa neda vybrat

Skontroluj:

- selectable == true,
- visible == true,
- ci je widget v poli View* children[],
- ci Group dostala spravny childCount.

List sa neda opustit

Zapni:

```
list.setExitOnBoundary(true);
```

A skupina by mala mat selectCyclicMode == false, ak chces hranice posuvat smerom k nadradenemu vyberu.

MessageBox nereaguje

Skontroluj:

- ci eventReader funguje aj v blokujucej slucke,
- ci sa generuje PRESS_BUTTON,
- ci nie je tlacidlo encoderu stale v stlacenom stave bez debounce.

Pages sa neprepina

Skontroluj:

- selectable == true,
- pageCount > 1,
- pageCallbacks != nullptr,
- ci je Pages vybrany v skupine.

Displej sa neprekresluje

Skontroluj:

- ci volas screenManager.update(),
 - ci je nastavena aktivna obrazovka,
 - ci sa po zmene vola invalidate().
-

24. API referencia v skratke

Typy

```
enum class Event;  
typedef void (*PressCallback)(View* sender);  
typedef bool (*EventReader)(Event& event);  
enum class MessageBoxResult;  
enum class MessageBoxButtonType;  
struct MessageBoxButton;  
typedef void (*PageDrawCallback)(U8G2& u8g2, uint8_t pageIndex);
```

View

```
View(int16_t x, int16_t y, int16_t w, int16_t h, bool selectable = false);  
virtual void draw(U8G2& u8g2) = 0;  
virtual bool handleEvent(Event event);  
void setParent(Group* parent);  
Group* getParent() const;  
virtual void setScreenManager(ScreenManager* manager);  
ScreenManager* getScreenManager() const;  
void invalidate();  
void setVisible(bool value);  
bool isVisible() const;  
void setSelectable(bool value);  
bool isSelectable() const;  
void setSelected(bool value);  
bool isSelected() const;  
void setEditing(bool value);
```



```

bool isEditing() const;
void setPressCallback(PressCallback callback);
void notifyPress();
int16_t getX() const;
int16_t getY() const;
int16_t getW() const;
int16_t getH() const;

```

Group

```

Group(int16_t x, int16_t y, int16_t w, int16_t h, View** children, uint8_t childCount, bool
selectCyclicMode = true);
void draw(U8G2& u8g2) override;
bool handleEvent(Event event) override;
View* getSelectedChild();
int8_t getSelectedIndex() const;
virtual bool selectFirst();
virtual bool selectLast();
virtual bool selectNext();
virtual bool selectPrevious();
bool setSelectedIndex(uint8_t index);
void setSelectCyclicMode(bool value);
bool getSelectCyclicMode() const;
void setScreenManager(ScreenManager* manager) override;

```

Screen a ScreenManager

```

Screen(const char* name, View** children, uint8_t childCount, bool selectCyclicMode =
false);
Screen(const char* name, int16_t x, int16_t y, int16_t w, int16_t h, View** children,
uint8_t childCount, bool selectCyclicMode = false);
const char* getName() const;
virtual void onEnter();
virtual void onExit();
virtual void onUpdate();

```

```

ScreenManager(U8G2& u8g2);
void setScreen(Screen* screen);
Screen* getActiveScreen() const;
bool handleEvent(Event event);
void draw();
void update();
void invalidate();
bool isInvalidated() const;

```

Widgety

```
Label(int16_t x, int16_t y, const char* text, const uint8_t* font);
Icon(int16_t x, int16_t y, int16_t w, int16_t h, const uint8_t* bitmap, bool selectable =
false);
CheckButton(int16_t x, int16_t y, bool checked = false);
Number(int16_t x, int16_t y, int16_t w, int16_t h, int value, int minValue, int maxValue,
const uint8_t* font);
List(int16_t x, int16_t y, int16_t w, int16_t h, const char** items, uint8_t itemCount, uint8_t
visibleItemsCount, const uint8_t* font);
Pages(int16_t x, int16_t y, int16_t w, int16_t h, PageDrawCallback* pageCallbacks, uint8_t
pageCount, bool cyclic = false, bool selectable = true);
```

MessageBox

```
MessageBox(int16_t x, int16_t y, int16_t w, int16_t h, const char* text, MessageBoxButton*
buttons, uint8_t buttonCount, const uint8_t* font);
MessageBoxResult getResult() const;
MessageBoxResult messageBox(U8G2& u8g2, const char* text, MessageBoxButton*
buttons, uint8_t buttonCount, EventReader eventReader, const uint8_t* font);
```

Záver

MiniGui poskytuje malý, prehľadný a rozsiriiteľný základ pre ovládanie embedded GUI cez encoder. V aktuálnej podobe je vhodný pre battery tester, nastavovacie menu, výsledkové obrazovky, modálne potvrdenia a viacstránkové zobrazovanie meraných hodnôt.